

EKORA BAZAAR

Engineering Architecture & Developer Onboarding Guide *Subtitle: "Understanding the System Before Writing Code"*

TABLE OF CONTENTS

1. Executive Overview
2. Technology Stack
3. High-Level Architecture
4. Project Folder Structure
5. Request → Response Data Flow
6. Database Architecture
7. Authentication & Authorization
8. User Types & Access Control
9. Product Architecture
10. Order Architecture
11. Payment Architecture
12. Shipping Architecture
13. Search Architecture
14. SEO & GEO Architecture
15. Analytics Architecture
16. Admin Architecture
17. Seller Architecture
18. Security Architecture
19. Environment Variables
20. Development Workflow
21. Database Migrations
22. Testing & Debugging
23. Common Code Patterns
24. "How to Add a Feature" Guide
25. Project Rules for Interns
26. Quick Reference

1. EXECUTIVE OVERVIEW

Welcome to **Ekora Bazaar**, a specialized B2B and B2C ecommerce marketplace designed for the craft and formulation industry.

What Ekora Bazaar is: It is a multi-vendor marketplace where users can purchase specialized ingredients (like waxes, molds, pigments) and formulations, while approved sellers can manage their own catalogs and orders.

What users can do:

- Browse and search a massive catalog of products.
- Read formulations and craft guides.
- Add products to a cart (both as a guest and a logged-in user).
- Checkout securely using Razorpay.
- Track their orders.

What sellers can do:

- Log in to a dedicated seller dashboard (</seller/dashboard>).
- View and manage their specific products and stock.
- View incoming orders that exclusively contain their products.
- Maintain their business and GST profile.

What admins can do:

- Approve/reject seller applications and product uploads.
- Manage order fulfillment, including assigning AWBs and generating labels via Shiprocket.

High-Level Flow: User opens website → Next.js frontend renders the UI (often static or server-rendered) → Server Actions / API Routes handle logic → Prisma communicates with PostgreSQL database → Response returns to the UI.

2. TECHNOLOGY STACK

Technology	Where Used	Why We Use It
Next.js (App Router)	Entire Frontend & Backend Framework	Provides React-based UI, Server Components (for fast page loads), and API routes all in one codebase.
React & Tailwind CSS	UI Components	React handles interactive elements. Tailwind CSS allows rapid, utility-first styling without writing custom CSS files.
Prisma	Database ORM	Instead of writing raw SQL strings, Prisma provides a typed, auto-completed TypeScript client to safely query our PostgreSQL database.
PostgreSQL	Primary Database	A robust relational database capable of handling complex relationships (Orders, OrderItems, Products, Sellers) and advanced fuzzy search.
Jose (JWT)	Authentication	We use stateless JWTs stored in HTTP-only cookies to verify user sessions without querying the database on every page load.
bcrypt	Password Security	securely hashes user passwords before storing them in the database, protecting against data breaches.
Razorpay	Payments	The standard for Indian payment processing, providing UPI, card, and net banking support.
Shiprocket	Logistics & Fulfillment	Automates courier assignment, label generation, and tracking updates.

Technology	Where Used	Why We Use It
Vercel	Hosting & Analytics	Vercel hosts our Next.js application at the edge and provides built-in GA4/Web Vitals analytics.
Zod	Validation	Provides strict schema validation to ensure incoming API data matches expected formats before touching the database.

3. HIGH-LEVEL ARCHITECTURE

The Ekora Bazaar application follows a modern monolithic architecture utilizing Next.js Server Components and Server Actions.

```
graph TD
    User([User Browser])
    NextJS[Next.js Application]
    Auth[Auth Layer - JWT/Jose]
    DB[(PostgreSQL via Prisma)]
    RZP[Razorpay API]
    Ship[Shiprocket API]

    User -->|HTTP Requests| NextJS
    NextJS -->|Session Cookies| Auth
    Auth -->|Validated Session| NextJS
    NextJS -->|Prisma Client| DB
    NextJS -->|Payment Verification| RZP
    NextJS -->|Fulfillment & AWBs| Ship
```

4. PROJECT FOLDER STRUCTURE

Understanding the folder structure is critical before writing code.

Folder / File Area	Responsibility	What Interns Should Know
src/app/	Frontend Routes	Contains all UI pages. Subfolders define the URL paths (e.g., <code>src/app/shop/page.tsx</code> maps to <code>/shop</code>).
src/app/api/	Backend API Endpoints	Contains Next.js Route Handlers (<code>route.ts</code>). This is where server-to-server endpoints (like webhooks) live.
src/app/admin/	Admin Portal	Protected routes for platform administrators.
src/app/seller/	Seller Portal	Protected routes for vendors. Code here is strictly scoped to <code>session.sellerId</code> .
src/app/account/	Customer Portal	Protected routes for buyers (order history, addresses).
src/components/	Reusable React UI	Shared buttons, product cards, navbars. Kept modular to avoid repeating code in pages.
src/lib/	Core Utilities	Shared logic (e.g., <code>auth.ts</code> , <code>db.ts</code> , <code>search.ts</code>). Highly sensitive business logic lives here.
prisma/	Database Config	Contains <code>schema.prisma</code> (our database blueprint) and the <code>migrations/</code> folder.

5. REQUEST → RESPONSE DATA FLOW

A. Normal Page Load (e.g., Homepage)

Browser requests `/` → Next.js Server renders the React components to HTML → Next.js sends the fully formed HTML to the browser (extremely fast, good for SEO).

B. Product Browsing

User requests `/products/123` → Next.js Server Component queries `prisma.product.findUnique()` → Data is embedded directly into the UI on the server → Browser receives the final product page.

C. Search Implementation

User types "mold" → UI sends query to `/api/products?q=mold` → `src/lib/search.ts` expands synonyms (e.g., adds "mould", "silicone mold") → API runs a raw parameterized `pg_trgm` fuzzy search in PostgreSQL → Ranked results are returned to the UI. *(Note: We use database-level search because filtering 10,000+ JSON items in memory would crash the server).*

D. Checkout Flow

1. Cart UI sends payload to `/api/checkout/create-order`.
2. **Server** validates actual prices and stock via Prisma (ignoring frontend prices to prevent tampering).
3. Server generates a `Razorpay Order ID` and an internal `Order` with `status: PAYMENT_PENDING`.
4. Browser opens Razorpay widget.
5. On success, browser sends Razorpay signature to `/api/checkout/confirm-payment`.
6. Server verifies the cryptographic signature, marks `Order` as `PAID`, and clears the cart.

E. Seller Order Flow

Customer order is saved as multiple `OrderItem` records. Each `OrderItem` natively stores `sellerId`. When Seller A logs into `/seller/dashboard/orders`, the server queries `prisma.orderItem.findMany({ where: { sellerId: session.sellerId } })`. Seller A *cannot* see items belonging to Seller B.

F. Shiprocket Fulfillment

Admin views paid order → Clicks "Create Shipment" → Server calls Shiprocket API → Shiprocket returns an AWB (Tracking Number) → Server updates `Order` shipping status. When the courier scans the package, Shiprocket fires a webhook to `/api/webhooks/shiprocket`, which automatically updates the UI for the customer.

6. DATABASE ARCHITECTURE

Our PostgreSQL database is managed entirely by Prisma.

```
erDiagram
    User ||--o{ Address : has
    User ||--o{ Order : places
    User ||--o{ Seller : can_be
    Seller ||--o{ Product : owns
    Order ||--|{ OrderItem : contains
    Product ||--o{ OrderItem : included_in
    Seller ||--o{ OrderItem : fulfills
```

Key Models:

- **User** : Handles authentication credentials, roles (**CUSTOMER** , **SELLER** , **ADMIN**), and contact info.
- **Seller** : Linked 1-to-1 with a User. Contains **brandName** , **accountStatus** , and connects to **SellerBusiness** .
- **Product** : Owned by a Seller. Tracks **price** , **stock** , and metadata.
- **Order** : Represents a customer's checkout session. Tracks total cost, payment status, and shipping address snapshot.
- **OrderItem** : Connects an **Order** to a **Product** . **Crucially**, it contains **priceSnapshot** and **productNameSnapshot** so that if a seller changes a price tomorrow, historical receipts don't break.

7. AUTHENTICATION & AUTHORIZATION

Authentication (Who are you?): When a user logs in, `bcrypt.compare()` verifies their password. If successful, `src/lib/session.ts` generates a cryptographically signed JWT using the `jose` library. This token is placed in an HTTP-only cookie, meaning JavaScript cannot read it (protecting against XSS attacks).

Authorization (What can you do?): `src/middleware.ts` acts as a security guard. If a user tries to visit `/admin/*`, the middleware intercepts the request, reads the JWT cookie, checks if `role === 'ADMIN'`, and redirects them if they fail the check. Similarly, `requireAuth()`, `requireSeller()`, and `requireAdmin()` in `src/lib/auth.ts` protect server components and APIs from unauthorized access.

8. USER TYPES & ACCESS CONTROL

Role	Can Access	Cannot Access
Customer	/shop, /cart, /account	/seller/*, /admin/*
Seller	/seller/dashboard/*, own products/orders	/admin/*, other sellers' data
Admin	/admin/*, all global platform data	N/A (Full access)

Seller Data Isolation: Server-side queries strictly enforce isolation. A seller fetching orders must use `where: { sellerId: session.sellerId }`. We *never* trust a `sellerId` passed from the frontend client.

9. PRODUCT ARCHITECTURE

Products are created by sellers and approved by admins (or automatically, depending on application status).

- **Ownership:** Every product enforces a `sellerId`.
- **Stock:** Handled transactionally. When an order is placed, stock is decremented atomically in the database (`decrement: quantity`) to prevent race conditions (two people buying the last item simultaneously).
- **Search:** The platform utilizes PostgreSQL `pg_trgm` GIN indexes to allow fast fuzzy searching across 10,000+ items.

10. ORDER ARCHITECTURE: SNAPSHOTS

Why do we have `Order` AND `OrderItem`? A customer might buy 5 different products in one cart. The `Order` represents the single payment and delivery address. The `OrderItem`s represent the individual line items.

The Snapshot Rule: `OrderItem` copies the product's price (`priceSnapshot`) and name (`productNameSnapshot`) at the exact moment of checkout. If you do not do this, changing a product's price from ₹500 to ₹600 tomorrow will magically rewrite yesterday's accounting receipts. **Never reference the live product price for historical orders.**

11. PAYMENT ARCHITECTURE

The Golden Rule of Payments: Never trust the frontend. A malicious user can modify frontend JavaScript to send `{ price: 1 }` to the server. Therefore, our checkout API (`/api/checkout/create-order`) completely ignores the frontend cart prices. It takes the requested `productId` s, queries the database for the *real* prices, calculates the total server-side, and requests that exact amount from Razorpay.

12. SHIPPING ARCHITECTURE

Currently implemented via Shiprocket:

1. **Admin Fulfillment:** Admins click "Create Shipment".
2. **Shiprocket API:** We post the package weight, dimensions, and customer address to Shiprocket.
3. **AWB Generated:** Shiprocket returns a tracking number (AWB).
4. **Webhooks:** When the package moves (e.g., "Out for Delivery"), Shiprocket automatically hits our `/api/webhooks/shiprocket` endpoint, which parses the event and updates our database.

13. SEO & GEO ARCHITECTURE

Ekora Bazaar relies heavily on organic search traffic.

- **Indexability:** We dynamically inject `<meta name="robots" content="noindex">` into private pages like `/cart` , `/checkout` , and `/account` to keep Google out.
- **Sitemap:** Automatically generated in `src/app/sitemap.ts` , ensuring Google crawls all products, categories, and public guides.
- **Metadata:** Handled natively by Next.js `generateMetadata` exports.

14. ANALYTICS ARCHITECTURE

We track e-commerce behavior using GA4. Important events include:

- `view_item` : Triggered on product pages.
- `add_to_cart` : Triggered when cart state changes.
- `begin_checkout` : Tracks intent to buy.
- `purchase` : Tracks completed, verified revenue.

These metrics allow the business to calculate conversion rates and cart abandonment.

15. PROJECT RULES FOR INTERNS

Read these carefully. Violating these rules can compromise production data.

1. **Understand before modifying.** Do not copy-paste code if you do not understand the data flow.
2. **Never guess database relationships.** Always read `prisma/schema.prisma` first.
3. **Never trust the frontend.** Always validate inputs, prices, and IDs on the server.
4. **Never expose secrets.** Do not commit `.env` files or console.log sensitive API keys.
5. **Never use `prisma db push` in production.** Always use `npx prisma migrate dev` locally to generate standard migration files, which are then applied safely.
6. **Never delete production data to fix a UI bug.** If a page crashes because data is missing, write robust code to handle the missing data (e.g., optional chaining `?.`), do not delete the "broken" data.
7. **Test failures, not just successes.** What happens if the payment fails? What if stock is 0?

16. DEVELOPMENT WORKFLOW

1. Clone the repository and run `npm install`.
2. Ask your manager for the local `.env.local` file (containing test database strings and development API keys).
3. Run `npm run dev` to start the Next.js server.
4. Make your changes in a new branch (e.g., `feature/add-wishlist`).
5. Run `npx tsc --noEmit` to ensure you have no TypeScript errors.
6. Run `npm run build` to ensure the production build succeeds.
7. Push your branch and open a Pull Request.

17. TESTING & DEBUGGING CHECKLIST

If a page throws a 500 "Something went wrong" error:

1. Check the server terminal (where `npm run dev` is running) for the exact stack trace.
2. If it's a Client Component, check the Browser DevTools console.
3. Check the API response in the Network tab.
4. Are you logged in? Check if your cookies contain a valid `session`.
5. Did you access a null relationship? (e.g., `item.order.status` crashing because `item.order` is missing).
6. Never suppress an error without understanding its root cause.

18. QUICK REFERENCE CHEAT SHEET

Architecture: Frontend → Server Actions/API → Prisma → PostgreSQL

Important Folders: `src/app` (Routes), `src/lib` (Logic), `prisma` (Database).

Commands:

- Start Dev: `npm run dev`
- Typecheck: `npx tsc --noEmit`
- Test Build: `npm run build`
- DB Studio: `npx prisma studio`

Welcome to the team, and happy coding!